样题1

模块一：平台搭建与运维

(一)任务一：大数据平台搭建

1.子任务一：基础环境准备

(1)`ifconfig` 查看ip

`hostnamectl set-hostname master` 配置主机名

`systemctl restart network` 重启服务

`vim /etc/hosts` 打开配置文件

```
ip master
ip slave1
ip slave2
```

`cat /etc/hosts`

(2)`mkdir -p /opt/module` 创建文件夹

`tar -zxvf /opt/software/jdk-8u191-linux-x64.tar.gz -C /opt/module`

若 slave1、slave2节点不存在以上文件则需从master节点复制

`scp -r /opt/module/jdk root@slave1:/opt/module`

`scp -r /opt/module/jdk root@slave2:/opt/module`

(3)`vim /etc/profile` 配置文件

```
export JAVA_HOME=/opt/module/jdk
export PATH=$PATH:$JAVA_HOME/bin
```

`source /etc/profile` 立即生效

(4)`java -version`

(5)`useradd hadoop` 创建用户hadoop

`passwd hadoop` 设置密码

`visudo /etc/sudoers` 配置管理员

```
hadoop ALL=(ALL)    NOPASSWD:ALL
```

`grep 'hadoop' /etc/sudoers`

(6)`systemctl stop firewalld` 关闭防火墙

`systemctl disable firewalld` 禁止开机自启

`systemctl status fireawalld` 查看状态

(7)`ssh-keygen -t rsa` 生成密钥

`ssh-copy-id hadoop@slave1` 复制密钥到其他节点

`ssh-copy-id hadoop@slave2`

`sudo systemctl restart sshd` 重启ssh服务

`ssh slave1(slave2)` 尝试连接

2.子任务二：Hadoop 完全分布式安装配置

(1)`tar -zxvf /opt/software/hadoop-3.3.6.tar.gz -C /opt/module`

(2)`mv /opt/module/hadoop-3.3.6 /opt/module/hadoop` 重命名

`chown -R hadoop:hadoop /opt/module/hadoop` 修改所属主/组

(3)`cd /opt/module/hadoop/etc/hadoop`

hadoop-env.sh

```shell
export JAVA_HOME=/opt/module/jdk
export HADOOP_HOME=/opt/module/hadoop
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop
export HADOOP_LOG_DIR=${HADOOP_HOME}/logs
```

core-site.xml

```xml
<property>
   <name>fs.defaultFS</name>
   <value>hdfs://master:9820</value>
</property>
<property>
   <name>hadoop.tmp.dir</name>
   <value>/opt/module/hadoop/tmp</value>
</property>
```

hdfs-site.xml

```xml
<property>
   <name>dfs.replication</name>
   <value>3</value>
</property>
<property>
   <name>dfs.namenode.secondary.http-address</name>
   <value>master:9896</value>
</property>
<property>
   <name>dfs.datanode.data.dir</name>
   <value>/opt/module/hadoop/datanode/data</value>
</property>
```

mapred-site.xml

```xml
<property>
   <name>mapreduce.framework.name</name>
   <value>yarn</value>
</property>
<property>
   <name>mapreduce.jobhistory.address</name>
   <value>master:10020</value>
</property>
<property>
   <name>mapreduce.jobhistory.webapp.address</name>
   <value>master:19888</value>
</property>
```

yarn-site.xml

```xml
<property>
   <name>yarn.resourcemanager.hostname</name>
   <value>master</value>
</property>
<property>
   <name>yarn.nodemanager.aux-services</name>
   <value>mapreduce_shuffle</value>
</property>
<property>
   <name>yarn.log-aggregation-enable</name>
   <value>true</value>
</property>
<property>
   <name>yarn.log.server.url</name>
   <value>http://master:19888/jobhistory/logs</value>
</property>
```

masters

```
master
slave1
slave2
```

workers

```
master
slave1
slave2
```

(4)`scp -r /opt/module/hadoop hadoop@slave1:/opt/module` 发送到其它节点
`scp -r /opt/module/hadoop hadoop@slave2:/opt/module`
(5)`chown -R hadoop:hadoop /opt/module/hadoop` 其余两个节点hadoop 安装目录的所有文件修改所属组者
(6)`vim /etc/profile` 配置环境

```
export HADOOP_HOME=/opt/module/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

`source /etc/profile` 立即生效
(7)`su - hadoop` 切换到hadoop用户
`hdfs namenode -format` 初始化Hadoop环境namenode
日志最后二十行.....

(8)`start-dfs.sh` 启动hdfs
`start-yarn.sh` 启动yarn
`mapred --daemon start historyserver` 启动历史服务
`jps` 查看java进程
结果...
(9)`ssh slave1` 切换节点
`jps` 查看java进程
结果...

3.子任务三：Zookeeper 集群安装配置

(1)`tar -zxvf /opt/software/apache-zookeeper-3.8.3bin.tar.gz -C /opt/module`
(2)`mv /opt/module/apache-zookeeper-3.8.3-bin /opt/module/zookeeper-3.8.3`
(3)`vim /etc/profile`

```
export ZOOKEEPER_HOME=/opt/module/zookeeper-3.8.3
export PATH=$PATH:$ZOOKEEPER_HOME/bin
```

`source /etc/profile`
(4)`cd /opt/module/zookeeper-3.8.4/conf`
`mv zoo_sample.cfg zoo.cfg` 重命名
`vim zoo.cfg`
添加以下内容

```
server.1=master:2888:3888
server.2=slave1:2888:3888
server.3=slave2:2888:3888
```

(5)`cd /opt/module/zookeeper-3.8.4` 切换目录
`mkdir data` 创建文件夹data
`vim myid` 在文件里添加1
(6)`scp -r /opt/module/zookeeper-3.8.4 root@slave1:/opt/module`
`scp -r /opt/module/zookeeper-3.8.4 root@slave2:/opt/module`
(7)`vim myid`
slave1修改为2，slave2修改为3
(8)`zkServer.sh start` 启动zookeeper
(9)`zkServer.sh status` 查看状态
(10)`jps` 查看java进程

4.子任务四：Kafka 安装配置

(1)`tar -xvf /opt/software/kafka_2.12-3.6.1.tgz -C /opt/module` 解压
`mv /opt/module/kafka_2.12-3.6.1 /opt/module/kafka` 重命名
(2)上一题Zookeeper
(3)`vim /etc/profile`

```
export KAFKA_HOME=/opt/module/kafka
export PATH=$PATH:$KAFKA_HOME/bin
```

(4)`kafka-server-start.sh --version` 查看版本
截图略
(5)`cd /opt/module/kafka/config`
`vim sever.properties` 打开配置文件
`broker.id=1` 修改
`zookeeper.connect=master:2181,slave1:2181,slave2:2181/kafka` 配置连接Zookeeper集群地址
`scp -r /opt/module/kafka root@slave1:/opt/module` 分发
`scp -r /opt/module/kafka root@slave2:/opt/module`
`broker.id=2`slave1节点修改
`broker.id=3`slave2节点修改
`kafka-server-start.sh -daemon server.properties` 启动服务
(6)`kafka-topics.sh --bootstrap-server master:9092 --create --topic installtopic --partitions 2 --replication-factor 2` 创建Topic,名称为installtopic,分区数为2,副本数为2

(二)任务二：数据库服务器的安装与运维

1.子任务一：MySQL 安装配置

(1)`tar -xvf /opt/module/mysql-5.7.44-1.el7.x86_64.rpm-bundle.tar -C /opt/module`
(2)`rpm -ivh mysql-community-common mysql-community-libs mysql-community-libs-compat mysql-community-client mysql-community-server`
(3)`systemctl start mysqld` 启动MySQL
`systemctl enable mysqld` 设置开机自启
`mysqld --initialize --user=mysql` 初始化设置

2.子任务二：MySQL 运维

(1)`mysql -version` 查看版本
(2)`mysql -u root -p` 登录MySQL
`create database house_market default character set utf8mb4;` 创建数据库
(3)`create user 'house_admin'@'%' identified by '密码'` 创建用户
`create user 'house_viewer'@'%' identified by '密码'`

`grant all privileges on house_market.* to 'house_admin'@'%';`
`grant select on house_market.* to 'house_viewer'@'%'`
`flush privileges;` 刷新权限
(4)备份脚本

```shell
cat > /.../backup.sh << 'EOF'
#!/bin/bash
user="root"
pass="zjm5404"
backup='/../..' # 备份路径
date=$(date +%Y%m%d_%H%M%S)
database="house_market"

mkdir -p "$backup" # 创建目录

mysqldump -u"$user" -p"$pass" "$database" > "${backup}/${database}_backup_$date.sql"

find "$backup" -name "${database}_backup_*.sql" -mtime +7 -delete
EOF

chmod +x /.../backup.sh # 添加脚本执行权限

(crontab -l 2>/dev/null; echo "0 2 * * * /.../backup.sh") | crontab -
crontab -l # 验证cron任务
```

(5)`show variables like 'max_connections';` 查看最大连接数
`show variables like 'query_cache_size';` 查看缓存大小
(6)`vim /etc/my.cnf`

```
[mysqld]
max_connections = 1000
query_cache_size = 64M
```

`systemctl restart mysqld` 重启MySQL
`show variables like 'max_connections';`
`show variables like 'query_cache_size';`
(7)`use house_market;`
`show engines;` 查看存储引擎类型
(8)`create user 'backup_user'@'%' inedtified by '密码';`
`grant backup_admin,reload on *.* to 'backup_user'@'%';`
`flush privileges;`

3.子任务三：数据表的创建及维护

(1)`use house_market;`

```
create table house_info (
   house_id int primary key,
   community varchar(100),
   district varchar(50),
   layout varchar(50),
   area decimal(10,2),
   price decimal(12,2),
   floor_info varchar(50),
   orientation varchar(50),
   status enum('在售','已售') default '在售'
);
```

(2)

```
create table community_info(
   community_id int,
   community_name varchar(100),
   district varchar(50),
   build_year year,
   property_fee decimal(10,2),
   subway_distance int,
   primary key(community_id)
);
```

(3)

```
create table transaction_records(
   record_id int,
   house_id int,
   transaction_date date,
   transaction_price decimal(12,2),
   price_change decimal(12,2),
   transaction_type varchar(50),
   primary key(record_id)
);
```

(4)统计每个区域的在售房源数量和平均价格

```
select
   district,
   count(*),
   avg(price) as 平均价格
from
   house_info
where
   status = '在售'
group by
    district;
```

查询成交价格高于该区域平均成交价格的房源信息

```
select
   h.*,
   t.*
from
   house_info as h
join
   transaction_records as t
on
   h.house_id = t.house_id
where
   t.transaction_price > (
   select
       avg(t2.transaction_price)
   from
       transaction_records as t2
   join
       house_info as h2
   on
       t2.house_id = h2.house_id
   where h2.district = h.district
   );
```

查询距离地铁站1000米以内的小区及其房源数量

```
select
   c.community_name,
   count(h.house_id) as 房源数量
from
   house_info as h
join
   community_info as c
on
   c.community_name = h.community
where
   c.subway_distance < 1000
group by
   c.community_name;
```

统计每种户型的平均单价（按面积计算）并按降序排列

```
select
   h.layout
   avg(h.price/h.area) as 平均单价
from
   house_info as h
where
   h.area > 0
group by
   h.layout
order by
   平均单价 desc;
```

查询最近一个月内成交的房源信息及其所在小区详情

```
select
   t.*,
   h.*,
   c.*
from
   transaction_records as t
join
   house_info as h
on
   t.house_id = h.house_id
join
   community_info as c
on
   h.community = c.community_name
where
   t.transaction_date >=subdate(curdate(),interval 1 month);
```

模块二：数据获取与处理

(一)任务一：数据获取与清洗

1.子任务一：数据获取

```
import pandas as pd
house_sales = pd.read_csv('house_sales.csv')
print(house_sales.head(10))
```

2.子任务二：使用 Python 进行数据清洗

(1)删除面积为空或为零的记录，并将结果存储为 `cleaned_data_c1_4.csv`，N 为删除的数据条数
(2)删除价格(万元)为空或异常高（超过平均价格的 3 倍）的记录，并将结果存储为 `cleaned_data_c2_6.csv`，N 为删除的数据条数
(3)对房型数据进行标准化，将不规范的房型描述「2房间2卫」转换为标准格式「2室0厅」，并将结果存储为 `cleaned_data_c3_0.csv`，N 为修改的数据条数（本数据集的 200 条原始记录中没有出现「2房间2卫」这一写法，实际修改条数为 0）
(4)删除电梯字段为空白的记录，并将结果存储为 `cleaned_data_c4_18.csv`，N 为删除的数据条数
(5)删除面积(㎡)小于 20 的记录，并将结果存储为 `cleaned_data_c5_3.csv`，N 为删除的数据条数

清洗过程行数变化：原始数据 200 条 → c1 后 196 条 → c2 后 190 条 → c3 后 190 条 → c4 后 172 条 → c5 后 169 条。

(二)任务二：数据标注

1.子任务一：价格区间标注

以每个市区的平均售价为基准，把房源划分为经济型、中档型和高端型，结果存入 `price_range_mark.csv`（169 行，列：编号、市区、小区、户型、朝向、楼层、装修情况、电梯、面积(㎡)、价格(万元)、年份、价格区间）。标注结果分布为：经济型 50 条、中档型 70 条、高端型 49 条，中档型房源占四成以上，是市场的主力需求区间。

2.子任务二：市区热门度标注

以全市平均房源数量（16.9 条）为基准标注各市区热门度，结果存入 `area_popularity_mark.csv`（169 行，在上一节各列基础上新增「区域热门度」列）。标注结果分布为：中热门 123 条、高热门 24 条、低热门 22 条；高热门的 24 条全部来自西城（24 条，超过全市均值的 120% 即 20.28 条），低热门的 22 条来自朝阳（11 条）与大兴（11 条），低于全市均值的 80% 即 13.52 条，其余七个市区均为中热门。

(三)任务三：数据统计

1.子任务一：HDFS 文件上传下载

(1)`hdfs dfs -mkdir /file2_1`

(2)`hdfs dfs -chmod 777 /file2_1`

(3)`hdfs dfs -get /file2_1 /tmp`

2.子任务二：计算输入文件中的单词数

(1)`hdfs dfs -mkdir -p /user/hadoop/input`

(2)`hdfs dfs -put /var/log/dmesg /user/hadoop/input`

(3)`cd /opt/module/hadoop/share/hadoop/mapreduce`
`hadoop jar hadoop-mapreduce-examples-3.3.6.jar wordcount /user/hadoop/input/dmesg/user/hadoop/output`

(4)`hdfs dfs -cat /user/hadoop/output/* | head -n 10`

3.子任务三：使用拟蒙特卡罗法估算 Pi 值

(1)`cd /opt/module/hadoop/share/hadoop/mapreduce`
`hadoop jar hadoop-mapreduce-examples-3.3.6.jar pi 16 10000`

(2)Pi 值估算结果如下（16 个 map，每个 map 处理 10000 个样本，总样本数 160000）

```
Estimated value of Pi is 3.1415xxxxx
```

估算公式为 `pi = 4 × 单位圆内点数 / 总样本数`，理论期望值 `pi ≈ 3.14159265358979`，实际运行结果在该值附近波动。

模块三：业务分析与可视化

(一)任务一：数据分析与可视化

1.子任务一：数据分析

(1)各市区二手房销售量倒序前三名：西城 24 套、石景山 19 套、海淀 19 套；全市房源主要集中在西城与城西、城中区域
(2)各户型平均售价倒序前三种：2室0厅1卫 1063.31 万元（14 套）、4房2卫 1049.85 万元（12 套）、3室3厅2卫 1028.21 万元（8 套）
(3)各市区平均售价倒序前三名：石景山 964.81 万元、房山 959.08 万元、通州 947.75 万元
(4)不同装修情况的房源数量正序：简装修 24 套、豪华装修 26 套、毛坯 35 套、中等装修 41 套、精装修 43 套
(5)出售价低于平均售价 876.71 万元且面积超过平均面积 116.62 ㎡ 的房源共 41 套，占全部房源的 24.26%，属于典型的「低总价、大面积」性价比房源

2.子任务二：数据可视化

(1)柱状图展示各市区的二手房销售数量，每个柱子代表一个市区，高度代表该市区的房源销售数量，横向对比十个市区的存量差异
(2)条形图比较不同户型的平均售价，横轴表示平均售价，纵轴表示户型，从高到低排列以便比较各户型定价水平
(3)折线图展示不同年份的二手房平均售价趋势，横轴为年份（2000-2024），纵轴为当年平均售价，反映价格随时间的走势
(4)散点图探索售价与面积之间的关系，横轴为售价、纵轴为面积，每个点代表一套房源，用于观察价格与面积是否存在相关性

(二)任务二：业务分析

1.平均售价：根据计算，数据集中所有房源的平均售价为876.71万元
2.最受欢迎的户型：根据销售量，确定户型3室2厅最受欢迎（17套）
3.高销量市区特征：销售量前25%的市区为西城、石景山、海淀、丰台（门槛 19.0 套），有以下特征
(1)价格范围在160.32 ~ 1484.23万元，价格平均数870.33万元
(2)面积范围在42.57(㎡) ~ 199.71(㎡)，平均面积119.33(㎡)
1.二手房市场的营销策略建议：根据用户的需求，优化房源供给，根据数据中最受欢迎的户型是3室2厅，建议中介和销售多推广和优先展示此类户型，满足大多数人的需求，同时也可以的其他的户型进行改造，将非标户型的调整为3室2厅的户型，以更好匹配市场需求。

2.针对分析的数据，加强市场推广力度。可以通过重点包装这些区域的房源的地段价值和生活便利性，并通过线上线下多渠道进行精准投放，吸引目标客户。